

Objetivos da revisão:

- Coesão e acoplamento
- Padrões de criação, estruturais e comportamentais
- Padrões arquiteturais MVC, MVP e MVVM

Questão 1 – Identificação de Coesão

Considere o código TypeScript a seguir:

```
class Sistema {
    salvarUsuario(nome: string) {
        console.log("Usuário salvo:", nome);
    }

    calcularSalario(valor: number) {
        return valor * 1.2;
    }

    enviarEmail(email: string) {
        console.log("Email enviado para:", email);
    }
}
```

Assinale a alternativa correta:

- a) A classe apresenta alta coesão, pois todos os métodos manipulam dados.
- b) A classe apresenta baixa coesão, pois possui múltiplas responsabilidades distintas.
- c) A classe apresenta baixo acoplamento, pois não depende de outras classes.
- d) A classe está correta, pois concentra funcionalidades comuns de um sistema.
- e) A classe apresenta alta coesão, pois todos os métodos pertencem à mesma classe.

Questão 2 – Aumento de Coesão

Considere a necessidade de melhorar o código a seguir:

```
class Pedido {
    adicionarProduto(nome: string) {}
    enviarEmailConfirmacao(email: string) {}
    gerarRelatorio() {}
}
```

Qual das alternativas representa a melhor abordagem para aumentar a coesão?

- a) Adicionar mais métodos à classe Pedido para centralizar funcionalidades.
- b) Criar subclasses herdando de Pedido para cada método.
- c) Separar as responsabilidades em classes específicas, como EmailService e RelatorioService.
- d) Tornar todos os métodos estáticos.

- e) Utilizar apenas funções globais no lugar da classe.

Questão 3 – Identificação de Alto Acoplamento

Analise o código:

```
class Logger {
    log(msg: string) {
        console.log(msg);
    }
}

class PedidoService {
    private logger = new Logger();

    finalizarPedido() {
        this.logger.log("Pedido finalizado");
    }
}
```

Assinale a alternativa correta:

- a) O código apresenta baixo acoplamento, pois Logger é reutilizável.
- b) O código apresenta alto acoplamento, pois PedidoService instancia diretamente Logger.
- c) O código apresenta alta coesão, logo o acoplamento não é relevante.
- d) O uso de classes elimina problemas de acoplamento.
- e) O acoplamento só ocorre quando há herança.

Questão 4 – Redução de Acoplamento

Considere a refatoração a seguir:

Código original:

```
class EmailService {
    enviar(email: string) {}
}

class UsuarioService {
    private emailService = new EmailService();
}
```

Código refatorado:

```
class UsuarioService {
    constructor(private emailService: EmailService) {}
}
```

Qual alternativa descreve corretamente a melhoria realizada?

- a) A coesão foi reduzida, pois a classe perdeu controle sobre suas dependências.

- b) O acoplamento aumentou, pois há mais parâmetros no construtor.
- c) Não houve mudança significativa na estrutura do sistema.
- d) A refatoração apenas melhora a performance do código.
- e) O acoplamento foi reduzido, pois a dependência é injetada externamente.

Questão 5 – Acoplamento em Componentes React

Considere o componente a seguir:

```
class UserService {
  getUser() {
    return { name: "João" };
  }
}

function UserProfile() {
  const userService = new UserService();
  const user = userService.getUser();

  return <div>{user.name}</div>;
}
```

Assinale a alternativa correta:

- a) O componente apresenta alto acoplamento, pois depende diretamente de UserService.
- b) O componente apresenta alta coesão, o que elimina problemas de acoplamento.
- c) O componente apresenta baixo acoplamento, pois utiliza uma classe separada.
- d) O código está ideal, pois concentra lógica e interface no mesmo local.
- e) O uso de TypeScript elimina o acoplamento entre componentes.

Questão 6 – Redução de Acoplamento com Props

Considere a refatoração do componente:

Versão refatorada:

```
type Props = {
  name: string;
};

function UserProfile({ name }: Props) {
  return <div>{name}</div>;
}
```

Assinale a alternativa correta:

- a) O componente ficou mais acoplado, pois depende de props.
- b) O componente ficou menos reutilizável.
- c) A coesão foi reduzida, pois o componente perdeu responsabilidade.

- d) O acoplamento foi reduzido, pois o componente não depende mais diretamente de serviços externos.
- e) A mudança não impacta o design do sistema.

Questão 7 – Inversão de Dependência (DIP)

Considere o código TypeScript a seguir:

```
class MySQLDatabase {
  salvar(dado: string) {
    console.log("Salvando no MySQL:", dado);
  }
}

class UsuarioService {
  private db = new MySQLDatabase();

  salvarUsuario(nome: string) {
    this.db.salvar(nome);
  }
}
```

Um desenvolvedor propõe a seguinte refatoração:

```
interface Database {
  salvar(dado: string): void;
}

class MySQLDatabase implements Database {
  salvar(dado: string) {
    console.log("Salvando no MySQL:", dado);
  }
}

class UsuarioService {
  constructor(private db: Database) {}

  salvarUsuario(nome: string) {
    this.db.salvar(nome);
  }
}
```

Assinale a alternativa correta:

- a) A refatoração aumenta o acoplamento, pois adiciona uma interface extra.
- b) A refatoração reduz a coesão da classe UsuarioService.
- c) A refatoração aplica o princípio da inversão de dependência, tornando o sistema mais flexível.
- d) A refatoração não altera o design, apenas a sintaxe do código.
- e) O uso de interface é desnecessário, pois classes concretas são mais eficientes.

Questão 8 – Padrão criacional

Considere o código:

```
class NotificacaoService {
    enviar(tipo: string) {
        if (tipo === "email") {
            console.log("Email enviado");
        } else if (tipo === "sms") {
            console.log("SMS enviado");
        }
    }
}
```

Qual problema esse código apresenta e qual padrão o resolve melhor?

- a) Alto acoplamento; solução: Singleton
- b) Baixa coesão; solução: Builder
- c) Falta de herança; solução: Prototype
- d) Excesso de interfaces; solução: Abstract Factory
- e) Lógica de criação misturada com uso; solução: Factory

Questão 9 – Padrão criacional

Analise o código:

```
class Logger {
    log(msg: string) {
        console.log(msg);
    }
}

const logger1 = new Logger();
const logger2 = new Logger();
```

Qual alternativa descreve corretamente o problema e a solução adequada?

- a) O código está correto, pois múltiplas instâncias aumentam a performance.
- b) Pode haver múltiplas instâncias desnecessárias; o padrão Singleton pode resolver.
- c) O problema é de herança; deve-se usar Factory.
- d) O problema é de coesão; deve-se usar Builder.
- e) O uso de classes elimina a necessidade de padrões.

Questão 10 – Padrão criacional

Considere:

```
class Usuario {
    constructor(
        public nome: string,
```

```
    public email: string,  
    public idade: number,  
    public endereco: string  
  ) {}  
}  
  
const usuario = new Usuario("Ana", "ana@email.com", 25, "Rua A");
```

Qual alternativa descreve corretamente o problema e a solução?

- a) O código apresenta baixo acoplamento; não precisa de alteração.
- b) O construtor é simples; Builder não se aplica.
- c) O problema é de instância única; usar Singleton.
- d) O problema é de herança; usar Prototype.
- e) Muitos parâmetros podem prejudicar a legibilidade; o Builder permite construção passo a passo.

Questão 11 – Padrão criacional

Considere o cenário:

Um sistema precisa criar componentes de interface para diferentes plataformas:

- Web: WebButton, WebInput
- Mobile: MobileButton, MobileInput

Qual alternativa descreve corretamente o uso do Abstract Factory?

- a) Criar uma única classe que instancia todos os objetos com if.
- b) Criar uma interface de fábrica que produza famílias de objetos relacionados (botão e input).
- c) Criar uma fábrica para cada objeto individual.
- d) Utilizar apenas herança entre classes concretas.
- e) Evitar interfaces para reduzir complexidade.

Questão 12 – Padrão estrutural

Considere o código:

```
class UsuarioService {  
  validarUsuario() {  
    console.log("Usuário válido");  
  }  
}  
  
class PagamentoService {  
  processarPagamento() {  
    console.log("Pagamento processado");  
  }  
}
```

```
class PedidoService {
  criarPedido() {
    console.log("Pedido criado");
  }
}

class CheckoutController {
  finalizar() {
    const usuario = new UsuarioService();
    const pagamento = new PagamentoService();
    const pedido = new PedidoService();

    usuario.validarUsuario();
    pedido.criarPedido();
    pagamento.processarPagamento();
  }
}
```

Qual alternativa apresenta a melhor justificativa para o uso do **Facade Pattern** nesse cenário?

- a) Substituir todas as classes por uma única classe grande.
- b) Criar uma classe que centralize o processo de finalização, reduzindo dependências do controlador.
- c) Utilizar herança entre os serviços para compartilhar comportamento.
- d) Remover a classe CheckoutController.
- e) Transformar os métodos em funções globais.

Questão 13 – Padrão Estrutural

Considere o código:

```
interface Pagamento {
  pagar(valor: number): void;
}

class SistemaPagamento implements Pagamento {
  pagar(valor: number) {
    console.log("Pagamento:", valor);
  }
}

class APIExterna {
  realizarPagamento(valor: number) {
    console.log("Pagamento externo:", valor);
  }
}
```

Deseja-se utilizar APIExterna sem modificar seu código.

Qual alternativa representa a melhor solução?

- a) Alterar a classe APIExterna para implementar Pagamento.

- b) Criar uma nova interface compatível com APIExterna.
- c) Criar uma classe adaptadora que implemente Pagamento e delegue para APIExterna.
- d) Substituir todas as chamadas por APIExterna diretamente.
- e) Utilizar herança múltipla entre as classes.

Questão 14 – Padrão Estrutural

Considere o código:

```
interface Notificacao {  
    enviar(msg: string): void;  
}  
  
class NotificacaoBasica implements Notificacao {  
    enviar(msg: string) {  
        console.log("Mensagem:", msg);  
    }  
}
```

Deseja-se adicionar funcionalidades como log e criptografia **sem modificar a classe original**.

Qual alternativa está correta?

- a) Modificar diretamente NotificacaoBasica para incluir todas as funcionalidades.
- b) Criar subclasses para cada variação possível de comportamento.
- c) Utilizar apenas funções externas para tratar a mensagem.
- d) Criar uma única classe com todas as funcionalidades possíveis.
- e) Utilizar decoradores que envolvem o objeto e adicionam comportamento dinamicamente.

Questão 15 – Padrão comportamental

Considere o código:

```
interface Observer {  
    update(valor: number): void;  
}  
  
class Sensor {  
    private observers: Observer[] = [];  
  
    adicionar(o: Observer) {  
        this.observers.push(o);  
    }  
  
    setValor(valor: number) {  
        for (const o of this.observers) {  
            o.update(valor);  
        }  
    }  
}
```



```
}
```

Qual alternativa descreve corretamente o padrão aplicado?

- a) Permite encapsular comandos em objetos independentes.
- b) Permite que múltiplos objetos sejam notificados quando ocorre uma mudança.
- c) Permite alterar o comportamento com base no estado interno.
- d) Permite executar uma cadeia de processamento sequencial.
- e) Permite escolher algoritmos em tempo de execução.

Questão 16 – Padrão comportamental

Considere o código:

```
interface Frete {  
    calcular(valor: number): number;  
}  
  
class FreteNormal implements Frete {  
    calcular(valor: number) {  
        return valor * 0.1;  
    }  
}  
  
class FreteExpresso implements Frete {  
    calcular(valor: number) {  
        return valor * 0.2;  
    }  
}  
  
class Pedido {  
    constructor(private frete: Frete) {}  
  
    total(valor: number) {  
        return this.frete.calcular(valor);  
    }  
}
```

Qual alternativa está correta?

- a) O padrão permite que o algoritmo de cálculo seja alterado sem modificar a classe Pedido.
- b) O padrão garante apenas uma instância da classe Frete.
- c) O código executa somente um estado.
- d) O padrão reduz o número de classes no sistema.
- e) O padrão exige o uso de herança múltipla.

Questão 17 – Padrão comportamental

Considere o código:

```
interface Command {
    executar(): void;
}

class Luz {
    ligar() {
        console.log("Ligada");
    }
}

class LigarCommand implements Command {
    constructor(private luz: Luz) {}

    executar() {
        this.luz.ligar();
    }
}

class Botao {
    constructor(private comando: Command) {}

    pressionar() {
        this.comando.executar();
    }
}
```

Qual alternativa descreve corretamente o benefício desse padrão?

- a) Elimina a necessidade de interfaces.
- b) Substitui completamente o uso de classes.
- c) Reduz o número de objetos no sistema.
- d) Permite desacoplar quem solicita a ação de quem a executa.
- e) Impede a reutilização de código.

Questão 18 – Padrão comportamental

Considere o código:

```
interface Estado {
    executar(): void;
}

class EstadoA implements Estado {
    executar() {
        console.log("Estado A");
    }
}
```

```
class EstadoB implements Estado {
    executar() {
        console.log("Estado B");
    }
}

class Contexto {
    constructor(private estado: Estado) {}

    executar() {
        this.estado.executar();
    }

    setEstado(estado: Estado) {
        this.estado = estado;
    }
}
```

Qual alternativa está correta?

- a) O comportamento do objeto muda conforme o estado interno.
- b) O padrão centraliza a criação de objetos.
- c) O padrão elimina completamente o uso de classes.
- d) O padrão impede mudanças de comportamento em tempo de execução.
- e) O padrão serve apenas para persistência de dados.

Questão 19 – Padrão comportamental

Considere o código:

```
abstract class Handler {
    protected proximo?: Handler;

    setProximo(h: Handler) {
        this.proximo = h;
    }

    abstract processar(valor: number): void;
}

class Nivel1 extends Handler {
    processar(valor: number) {
        if (valor < 100) {
            console.log("Nivel1 processou");
        } else if (this.proximo) {
            this.proximo.processar(valor);
        }
    }
}
```

Qual alternativa descreve corretamente esse padrão?

- a) Permite que apenas um objeto trate todas as requisições.
- b) Obriga o uso de herança profunda entre classes.
- c) Substitui completamente o uso de interfaces.
- d) Centraliza todas as decisões em uma única classe.
- e) Permite que múltiplos objetos tratem uma requisição em sequência.

Questão 20 – Classificação dos Padrões

Sobre padrões de projeto, assinale a alternativa correta:

- a) Padrões criacionais tratam da comunicação entre objetos.
- b) Padrões estruturais tratam da criação de objetos complexos.
- c) Padrões comportamentais tratam da forma como objetos interagem e colaboram.
- d) Todos os padrões de projeto têm o mesmo objetivo e não possuem classificação.
- e) Padrões estruturais eliminam a necessidade de orientação a objetos.

Questão 21 – Uso de Padrões no Design de Software

Considere a seguinte afirmação:

“Um sistema deve ser flexível, de fácil manutenção e permitir evolução sem grandes modificações no código existente.”

Qual alternativa melhor descreve o papel dos padrões de projeto nesse contexto?

- a) Substituir completamente a necessidade de programação.
- b) Fornecer soluções reutilizáveis para problemas recorrentes de design.
- c) Garantir que o código não utilize classes ou objetos.
- d) Reduzir o número de arquivos no sistema.
- e) Eliminar a necessidade de testes.

Questão 22 – Escolha de Padrões

Um sistema apresenta os seguintes problemas:

- Muitos if/else para escolher comportamentos
- Forte acoplamento entre classes
- Dificuldade para adicionar novas funcionalidades

Qual alternativa apresenta uma abordagem mais adequada?

- a) Utilizar apenas funções globais para simplificar o código.
- b) Aumentar o número de condicionais para cobrir novos casos.
- c) Centralizar toda a lógica em uma única classe.

- d) Remover todas as interfaces do sistema.
- e) Aplicar padrões como Strategy e Factory para reduzir acoplamento e melhorar extensibilidade.

Questão 23 – Identificação de Problema Arquitetural

Considere o código:

```
class SistemaUsuarios {  
    cadastrar(nome: string) {  
        console.log("Entrada do usuário");  
        console.log("Salvando no banco:", nome);  
        console.log("Atualizando interface:", nome);  
    }  
}
```

Assinale a alternativa correta:

- a) O código apresenta alta coesão e baixo acoplamento.
- b) O código está organizado em camadas bem definidas.
- c) O código aplica corretamente o padrão MVC.
- d) O código mistura responsabilidades de interface, lógica e dados.
- e) O código segue o princípio de separação de responsabilidades.

Questão 24 – Comparação entre MVC, MVP e MVVM

Assinale a alternativa correta sobre os padrões arquiteturais:

- a) No MVVM, a View reage automaticamente ao estado do ViewModel.
- b) No MVC, a View não acessa o Model diretamente.
- c) No MVP, a View acessa diretamente o Model.
- d) No MVC, o Presenter controla o fluxo da aplicação.
- e) No MVVM, o Controller é responsável pela lógica.

Questão 25 – Fluxo e Acoplamento

Considere as afirmações:

- I. No MVC, a View consulta diretamente o Model.
- II. No MVP, a View é passiva e depende do Presenter.
- III. No MVVM, a atualização da interface ocorre de forma reativa ao estado.

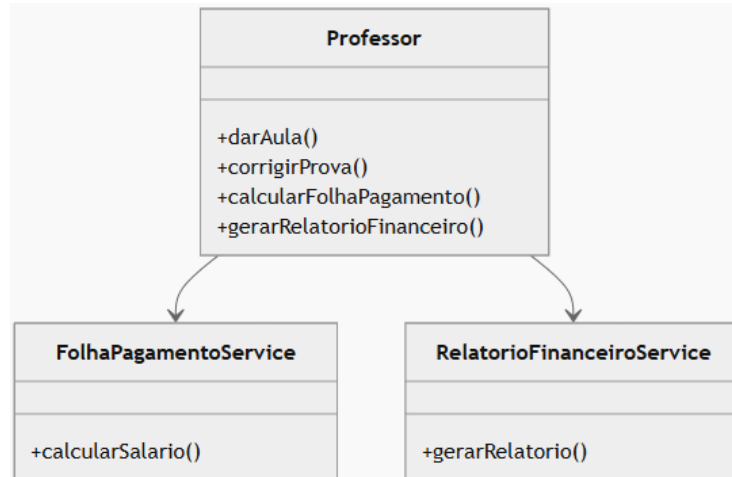
Assinale a alternativa correta:

- a) Apenas I está correta.
- b) Apenas I e II estão corretas.
- c) Apenas II e III estão corretas.
- d) I, II e III estão corretas.

e) Nenhuma está correta.

Questão 26 –Coesão e Acoplamento

Considere o diagrama UML a seguir:



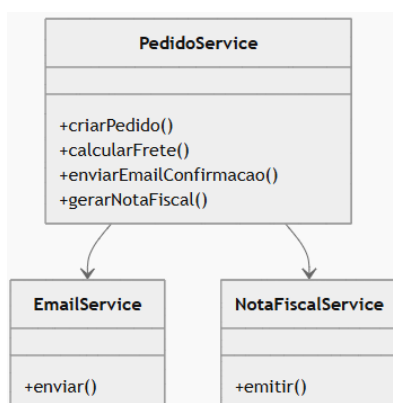
Com base no diagrama, assinale a alternativa correta:

- a) A classe Professor apresenta alta coesão, pois todas as operações estão relacionadas ao contexto educacional.
- b) A classe Professor apresenta baixa coesão, pois mistura responsabilidades acadêmicas e financeiras.
- c) O acoplamento é inexistente, pois as classes auxiliares estão separadas.
- d) O diagrama demonstra aplicação correta do princípio de responsabilidade única.
- e) A dependência com serviços externos reduz a necessidade de refatoração.

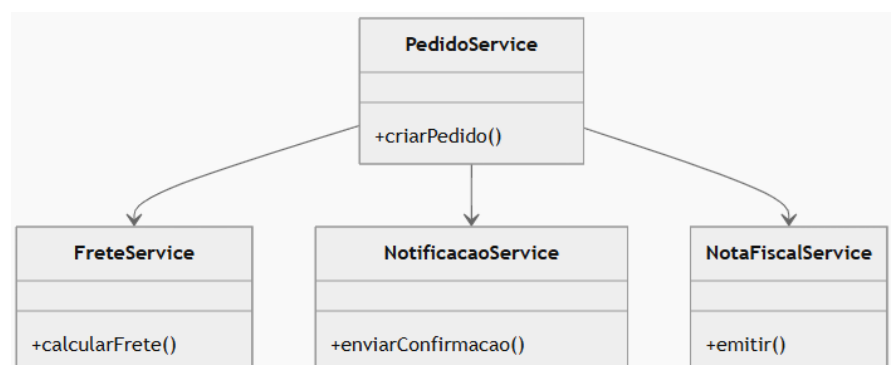
Questão 27 –Comparação de Diagramas UML (Antes vs. Depois)

Considere os dois diagramas a seguir:

Versão A (antes da refatoração)



Versão B (após refatoração)



Com base na análise dos diagramas, assinale a alternativa correta:

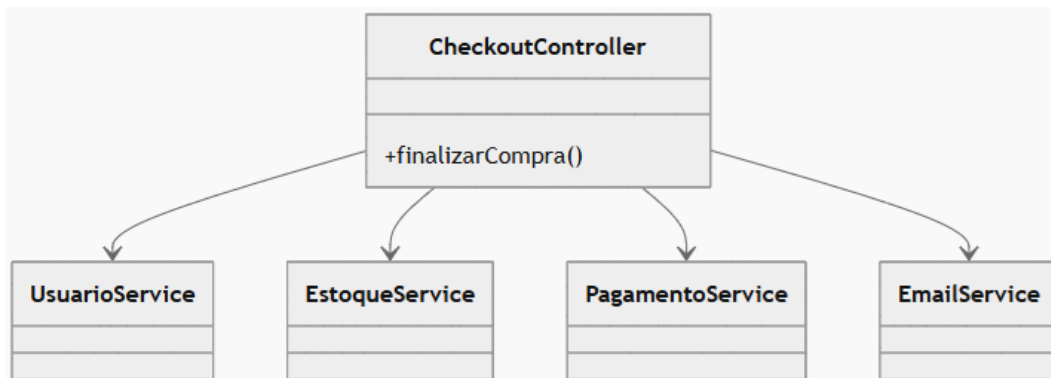
- a) A versão A possui maior coesão, pois concentra mais funcionalidades em uma única classe.
- b) A versão B apresenta menor acoplamento, pois remove dependências entre classes.
- c) A versão A apresenta melhor design, pois reduz o número de classes.

- d) Não há diferença significativa entre as duas versões.
- e) A versão B apresenta maior coesão e melhor separação de responsabilidades.

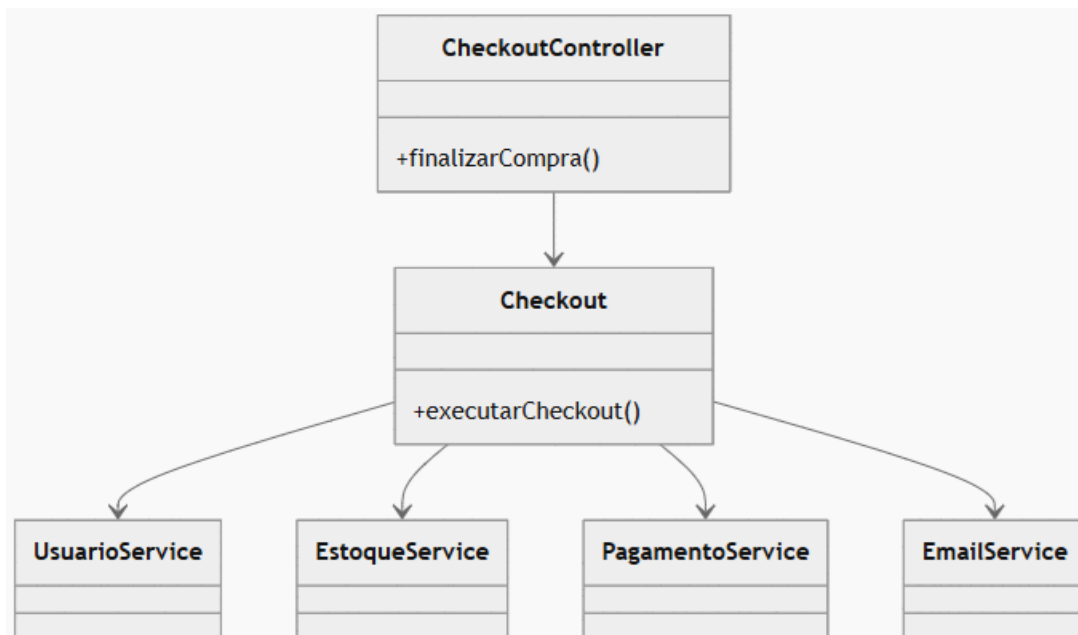
Questão 28 – Comparação de Diagramas UML (Antes vs. Depois)

Considere os dois diagramas a seguir:

Versão A (sem padrão)



Versão B (com refatoração)



Com base nos diagramas, assinale a alternativa correta:

- a) A versão A apresenta menor acoplamento, pois acessa diretamente os serviços.
- b) A versão B aplica o padrão Strategy para encapsular algoritmos.
- c) A versão B aplica o padrão Facade, reduzindo o acoplamento do controller com o subsistema.
- d) A versão A apresenta melhor coesão, pois evita classes intermediárias.
- e) A versão B aumenta o acoplamento geral do sistema sem benefícios.

Gabarito

Questão	Resposta
1	b
2	c
3	b
4	e
5	a
6	d
7	c
8	e
9	b
10	e
11	b
12	b
13	c
14	e
15	b
16	a
17	d
18	a
19	e
20	c
21	b
22	e
23	d
24	a
25	d
26	b
27	e
28	c